

Monitor de Discursos de Odio en Redes Sociales – Notebook EDA

Este notebook documenta el proceso de extracción, procesamiento y análisis exploratorio de tweets (en este caso desglosado por 20 usuarios) relacionados con discursos de odio en redes sociales, desarrollado como parte de un proyecto del CIAI (Centro de Investigación en Inteligencia Artificial de la UNSAM).

El objetivo del proyecto es monitorear y analizar la presencia y características de discursos de odio en la plataforma X (antes Twitter), generando información útil para investigaciones académicas y aplicaciones prácticas en la detección de contenido nocivo en redes sociales. Este análisis contribuye al desarrollo de herramientas y metodologías basadas en inteligencia artificial que permitan entender mejor cómo circula el discurso de odio y cómo se relaciona con eventos sociales y políticos.

```
import pandas as pd
import csv
import re
import matplotlib.pyplot as plt
import seaborn as sns
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from huggingface_hub import login
import os

# Leer CSV (METODO VISUALMENTE PREFERENTE)
tweets = pd.read_csv(
    "usuarios_tw_09_11_2025.csv",
    engine='python',          # motor más tolerante
    sep=',',                  # reconoce comillas
    quotechar='"',            # reconoce comillas
    quoting=csv.QUOTE_ALL,
    on_bad_lines='skip',      # ignorar filas corruptas
    encoding='utf-8'
)

# Visualizamos las primeras filas
tweets.head()
```

	id	date	text	username	user_displayname	user_id	reply_count	retweet_count	like_c
0	1986583511143489548	2025-11-07T02:56:32Z	Supermercados liquidan pequeños electrodomésticos...	todonoticias	TN - Todo Noticias	25992212	3	2	
1	1986583027133169735	2025-11-07T02:54:37Z	A un paso de la confirmación: el sugerente pos...	todonoticias	TN - Todo Noticias	25992212	0	1	
2	1986582923198538156	2025-11-07T02:54:12Z	José C. Paz: cómo sigue la salud de la nena qu...	todonoticias	TN - Todo Noticias	25992212	0	2	
3	1986580950310949294	2025-11-07T02:46:22Z	AHORA A un paso de la confirmación: el suger...	todonoticias	TN - Todo Noticias	25992212	1	8	
4	1986578201871618437	2025-11-07T02:35:26Z	Participó en "La Voz Argentina", lo eliminaron...	todonoticias	TN - Todo Noticias	25992212	1	3	

Próximos pasos:

[Generar código con tweets](#)

[New interactive sheet](#)

Analisis exploratorio de datos

```
# Información general
tweets.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 28640 entries, 0 to 28639
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    28640 non-null  int64
1   date                  28640 non-null  object
2   text                  28640 non-null  object
3   username              28640 non-null  object
4   user_displayname      28640 non-null  object
5   user_id               28640 non-null  int64
6   reply_count           28640 non-null  int64
7   retweet_count         28640 non-null  int64
8   like_count            28640 non-null  int64
9   quote_count           28640 non-null  int64
10  views_count           28640 non-null  int64
11  lang                  28640 non-null  object
12  url                   28640 non-null  object
13  user_followers        28640 non-null  int64
14  user_verified         28640 non-null  bool
15  nivel                 28640 non-null  int64
16  usuario_objetivo      28640 non-null  object
17  periodo               28640 non-null  object
dtypes: bool(1), int64(9), object(8)
memory usage: 3.7+ MB
```

El dataset contiene 28640 tweets extraídos del proceso de recolección. Las variables principales son:

- nivel: indica si el tweet fue emitido por el usuario objetivo (1) o dirigido a él (2).
- usuario_objetivo: cuenta analizada según el archivo de entrada.
- periodo: identifica si el tweet pertenece al rango histórico o al momento de ejecución.
- id / date / text: información básica del tweet (ID, fecha y contenido).
- username / user_displayname / user_id: datos de la cuenta que publicó el tweet.
- reply_count, retweet_count, like_count, quote_count, views_count: métricas de interacción.
- lang: idioma del tweet.
- url: enlace al tweet.
- user_followers / user_verified: características del usuario (seguidores y verificación).

```
tweets.describe(include='all')
```

	id	date	text	username	user_displayname	user_id	reply_count	retweet_count	like_c
count	2.864000e+04	28640	28640	28640	28640	2.864000e+04	28640.000000	28640.000000	28640.00
unique	NaN	25695	27026	12875	11991	NaN	NaN	NaN	
top	NaN	2025-11-06T15:07:44Z	🔥 "NO TE PREOCUPES CUANDO MANEJE CON UNA MANO"...	infobae	infobae	NaN	NaN	NaN	
freq	NaN	26	26	1206	1206	NaN	NaN	NaN	
mean	1.975736e+18	NaN	NaN	NaN	NaN	5.842185e+17	58.466236	94.922940	508.7
std	1.001329e+17	NaN	NaN	NaN	NaN	7.313047e+17	514.020665	768.334609	4184.66
min	1.762770e+09	NaN	NaN	NaN	NaN	7.812930e+05	0.000000	0.000000	0.00
25%	1.985774e+18	NaN	NaN	NaN	NaN	1.388140e+08	0.000000	0.000000	0.00
50%	1.986267e+18	NaN	NaN	NaN	NaN	1.615255e+09	0.000000	0.000000	1.00
75%	1.986548e+18	NaN	NaN	NaN	NaN	1.342863e+18	1.000000	1.000000	5.00
max	1.987594e+18	NaN	NaN	NaN	NaN	1.984396e+18	22637.000000	44252.000000	300578.00

Para un mejor manejo de los datos se procedio a convertir las variables de fecha en tipo de datos datetime

```
# Convertir columnas de texto a datetime
tweets['date'] = pd.to_datetime(tweets['date'], errors='coerce')
#tweets['fecha_captura'] = pd.to_datetime(tweets['fecha_captura'], errors='coerce')
```

tweets.dtypes

0	
id	int64
date	datetime64[ns, UTC]
text	object
username	object
user_displayname	object
user_id	int64
reply_count	int64
retweet_count	int64
like_count	int64
quote_count	int64
views_count	int64
lang	object
url	object
user_followers	int64
user_verified	bool
nivel	int64
usuario_objetivo	object
periodo	object

dtype: object

```
# Revisar primeras filas y tipos
tweets.head(5)
```

	id	date	text	username	user_displayname	user_id	reply_count	retweet_count	like_
0	1986583511143489548	2025-11-07 02:56:32+00:00	Supermercados liquidan pequeños electrodomésti...	todonoticias	TN - Todo Noticias	25992212	3	2	
1	1986583027133169735	2025-11-07 02:54:37+00:00	A un paso de la confirmación: el sugerente pos...	todonoticias	TN - Todo Noticias	25992212	0	1	
2	1986582923198538156	2025-11-07 02:54:12+00:00	José C. Paz: cómo sigue la salud de la nena qu...	todonoticias	TN - Todo Noticias	25992212	0	2	
3	1986580950310949294	2025-11-07 02:46:22+00:00	AHORA A un paso de la confirmación: el suger...	todonoticias	TN - Todo Noticias	25992212	1	8	
4	1986578201871618437	2025-11-07 02:35:26+00:00	Participó en "La Voz Argentina", lo eliminaron...	todonoticias	TN - Todo Noticias	25992212	1	3	

Próximos pasos: [Generar código con tweets](#) [New interactive sheet](#)

```
# Asegurar que la columna date sea datetime
tweets['date'] = pd.to_datetime(tweets['date'], errors='coerce')
```

```
# Extraer el año
tweets['year'] = tweets['date'].dt.year

# Contar tweets por año
tweets_por_año = tweets['year'].value_counts().sort_index()

print("Cantidad de tweets por año:")
print(tweets_por_año)
```

```
Cantidad de tweets por año:
year
2009      1
2010      1
2011     59
2020     13
2021    112
2022     38
2023    194
2024    143
2025   28079
Name: count, dtype: int64
```

✓ Limpieza y Preprocesamiento del Texto

Antes de realizar análisis sobre el contenido de los tweets, es necesario limpiar y preprocesar el texto para eliminar ruido y normalizarlo. Esto incluye:

- Eliminar caracteres especiales, URLs, menciones de usuarios y hashtags.
- Convertir el texto a minúsculas.
- Eliminar espacios en blanco adicionales.

```
def clean_tweet(tweet):
    if not isinstance(tweet, str):
        return ""

    tweet = re.sub(r'http\S+|www\S+|https\S+', '', tweet) # URLs
    tweet = re.sub(r'@\w+', '', tweet) # menciones
    tweet = re.sub(r'#\w+', '', tweet) # hashtags
    tweet = re.sub(r'[^A-Za-zñÁéíóúÁÉÍÓÜÜ\s]+', '', tweet) # símbolos/num
    tweet = tweet.lower()
    tweet = re.sub(r'\s+', ' ', tweet).strip() # espacios

    return tweet

tweets = tweets.dropna(subset=['text'])

tweets['tweet_processed'] = tweets['text'].apply(clean_tweet)

print(tweets[['text', 'tweet_processed']].head())
```

```
text \
0 Supermercados liquidan pequeños electrodomésti...
1 A un paso de la confirmación: el sugerente pos...
2 José C. Paz: cómo sigue la salud de la nena qu...
3 AHORA | A un paso de la confirmación: el suger...
4 Participó en "La Voz Argentina", lo eliminaron...

tweet_processed
0 supermercados liquidan pequeños electrodomésti...
1 a un paso de la confirmación el sugerente post...
2 josé c paz cómo sigue la salud de la nena que ...
3 ahora a un paso de la confirmación el sugerent...
4 participó en la voz argentina lo eliminaron an...
```

✓ Análisis de la Longitud de los Tweets

Exploraremos la longitud de los tweets antes y después del preprocesamiento para entender cómo varía y si hay tweets muy cortos o muy largos que puedan requerir un tratamiento especial.

```
# Longitud del tweet original
tweets['tweet_length'] = tweets['text'].astype(str).apply(len)

# Longitud del tweet procesado
tweets['tweet_processed_length'] = tweets['tweet_processed'].astype(str).apply(len)

# Estadísticas descriptivas
display(tweets[['tweet_length', 'tweet_processed_length']].describe())

# Tweet más largo y más corto (procesado)
max_length_tweet = tweets.loc[tweets['tweet_processed_length'].idxmax()]
min_length_tweet = tweets.loc[tweets['tweet_processed_length'].idxmin()]

print("\nTweet más largo (procesado):")
print(max_length_tweet['tweet_processed'])

print("\nTweet más corto (procesado):")
print(min_length_tweet['tweet_processed'])
```

	tweet_length	tweet_processed_length	
count	28640.000000	28640.000000	
mean	141.647661	113.218645	
std	197.557447	187.481455	
min	4.000000	0.000000	
25%	53.000000	35.000000	
50%	107.000000	79.000000	
75%	157.000000	124.000000	
max	8759.000000	8484.000000	

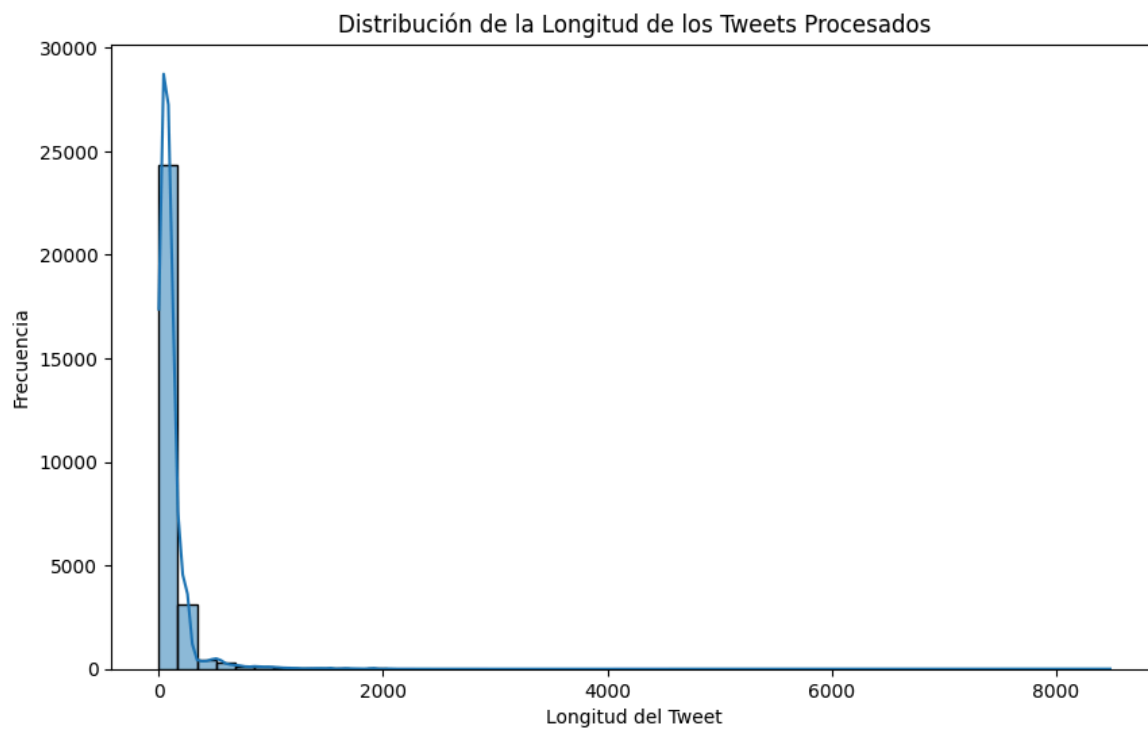
Tweet más largo (procesado):

axel kicillof acusó a javier milei de excluir gobernadores y cuestionó su acuerdo con donald trump el gobernador bonaerense pu

Tweet más corto (procesado):

Luego, para obtener mayor información se procedió a realizar un histograma. El cual brinda información estadística acerca de la longitud de los tweets. Este análisis sirve para obtener una idea inicial de la información que manejamos.

```
# Histograma de la longitud de los tweets procesados
plt.figure(figsize=(10, 6))
sns.histplot(tweets['tweet_processed_length'], bins=50, kde=True)
plt.title('Distribución de la Longitud de los Tweets Procesados')
plt.xlabel('Longitud del Tweet')
plt.ylabel('Frecuencia')
plt.show()
```



```
tweets.groupby(['usuario_objetivo', 'nivel']).size()
```

0		
usuario_objetivo	nivel	
A24COM	1	675
	2	1215
AmericaTV	1	415
	2	351
C5N	1	698
	2	1251
CFKArgentina	1	86
	2	1245
CasaRosada	1	21
	2	190
JMilei	1	349
	2	1336
LANACION	1	492
	2	1243
SergioMassa	1	211
	2	109
TNTSportsAR	1	670
	2	1218
canal26	1	60
	2	226
clarincom	1	604
	2	1220
cronica	1	773
	2	749
eldestape	1	1
	2	28
infobae	1	1200
	2	1218
mauriciomacri	1	177
	2	1326
pagina12	1	379
	2	1225
perfilcom	1	937
	2	611
radiomitre	1	611
	2	1233
todonoticias	1	1200
	2	1245
tycsports	1	621
	2	1221

dtype: int64

Descripción del Gráfico

El histograma muestra la distribución de la longitud de los tweets después del preprocesamiento. Se observa una concentración de tweets de longitud media, con algunos tweets más cortos y otros más largos. Esto nos da una idea de la variabilidad en el tamaño de los mensajes en el conjunto de datos. La mayoría de los tweets procesados se encuentran en el rango de aproximadamente 40 a 160 caracteres.

▼ Usuarios más activos

Ahora identificaremos a los usuarios que han publicado la mayor cantidad de tweets en el dataset.

```
usuarios_mas_activos = (
    tweets['username']
    .value_counts()
    .head(10)
)

print("Usuarios más activos:")
display(usuarios_mas_activos)
```

Usuarios más activos:

	count
username	
infobae	1206
todonoticias	1202
perfilcom	1084
cronica	821
C5N	718
TNTSportsAR	684
A24COM	675
TyCSports	622
radiomitre	613
clarincom	610

dtype: int64

▼ Cantidad de Tweets por Término de Discurso de Odio

Ahora, vamos a contar cuántos tweets procesados contienen cada uno de los términos de discurso de odio que has especificado.

```
# Asegurar que 'date' sea datetime
tweets['date'] = pd.to_datetime(tweets['date'], errors='coerce')

# Extraer hora y día correctamente
tweets['tweet_hour'] = tweets['date'].dt.hour
tweets['tweet_day'] = tweets['date'].dt.day_name()

# Contar tweets por día
tweets_per_day = tweets['tweet_day'].value_counts()

# Contar tweets por hora
tweets_per_hour = tweets['tweet_hour'].value_counts().sort_index()

print("Cantidad de tweets por día:")
display(tweets_per_day)

print("\nCantidad de tweets por hora:")
display(tweets_per_hour)
```


Cantidad de tweets por día:

	count
tweet_day	
Thursday	8481
Wednesday	5082
Friday	4089
Sunday	3295
Tuesday	2677
Saturday	2621
Monday	2395

dtype: int64

Cantidad de tweets por hora:

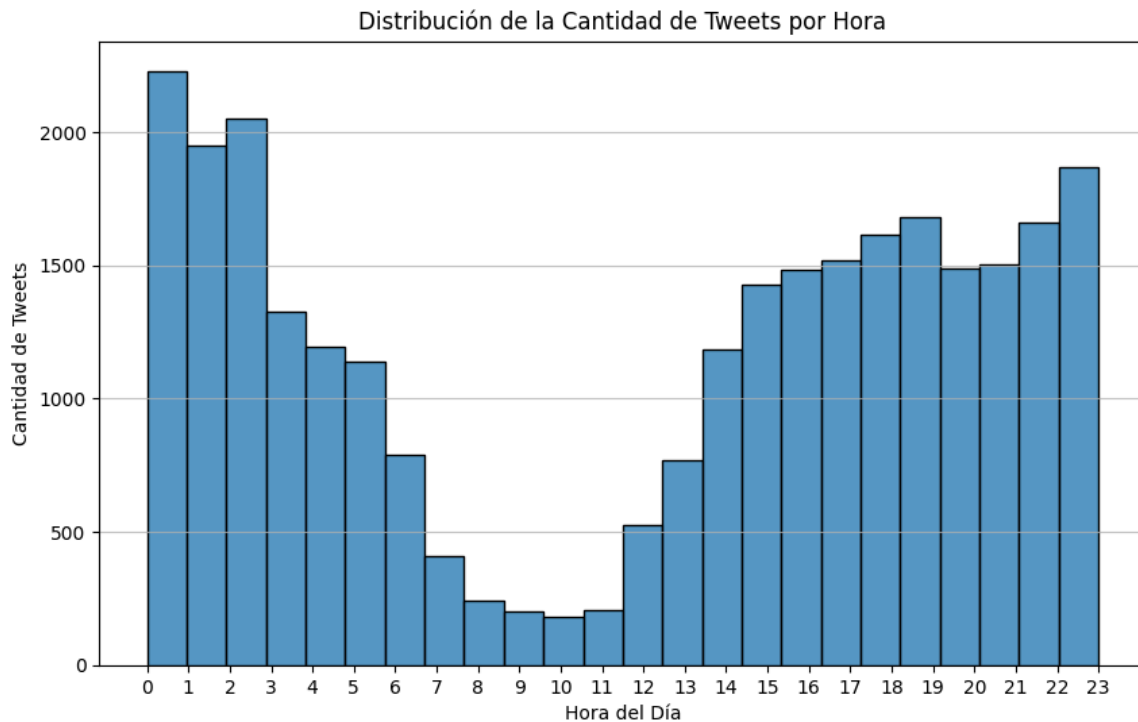
	count
tweet_hour	
0	2229
1	1948
2	2050
3	1325
4	1194
5	1140
6	790
7	408
8	244
9	199
10	179
11	208
12	525
13	771
14	1185
15	1429
16	1481
17	1518
18	1614
19	1683
20	1487
21	1505
22	1660
23	1868

dtype: int64

✓ Descripción del Gráfico de Distribución de Tweets por Hora

Este histograma nos muestra visualmente la concentración de tweets a lo largo de las 24 horas del día. Como habíamos anticipado por los conteos, el gráfico debería reflejar la alta actividad en el rango horario de 22 a 23 horas, mostrándonos exactamente cómo se distribuyen los tweets dentro de esa ventana de tiempo. Esto nos ayuda a confirmar y entender mejor los períodos de mayor publicación en este dataset.

```
# Histograma de la distribución de tweets por hora
plt.figure(figsize=(10, 6))
sns.histplot(tweets['tweet_hour'], bins=24, kde=False) # Usamos 24 bins para cada hora
plt.title('Distribución de la Cantidad de Tweets por Hora')
plt.xlabel('Hora del Día')
plt.ylabel('Cantidad de Tweets')
plt.xticks(range(24)) # Asegurar que se muestren todas las horas si hay datos
plt.grid(axis='y', alpha=0.75)
plt.show()
```



Visualización Temporal de los Tweets por Hora

Para tener una idea más clara de en qué momentos del día se publican más tweets, vamos a visualizar la cantidad de tweets por hora en un gráfico.

```
# Asegurar que la columna 'date' sea datetime
tweets['date'] = pd.to_datetime(tweets['date'], errors='coerce')

# Extraer la hora y el día correctamente
tweets['tweet_hour'] = tweets['date'].dt.hour
tweets['tweet_day'] = tweets['date'].dt.day_name()

# Contar tweets por día
tweets_per_day = tweets['tweet_day'].value_counts()

# Contar tweets por hora
tweets_per_hour = tweets['tweet_hour'].value_counts().sort_index()

print("Cantidad de tweets por día:")
display(tweets_per_day)

print("\nCantidad de tweets por hora:")
display(tweets_per_hour)
```

Cantidad de tweets por día:

	count
tweet_day	
Thursday	8481
Wednesday	5082
Friday	4089
Sunday	3295
Tuesday	2677
Saturday	2621
Monday	2395

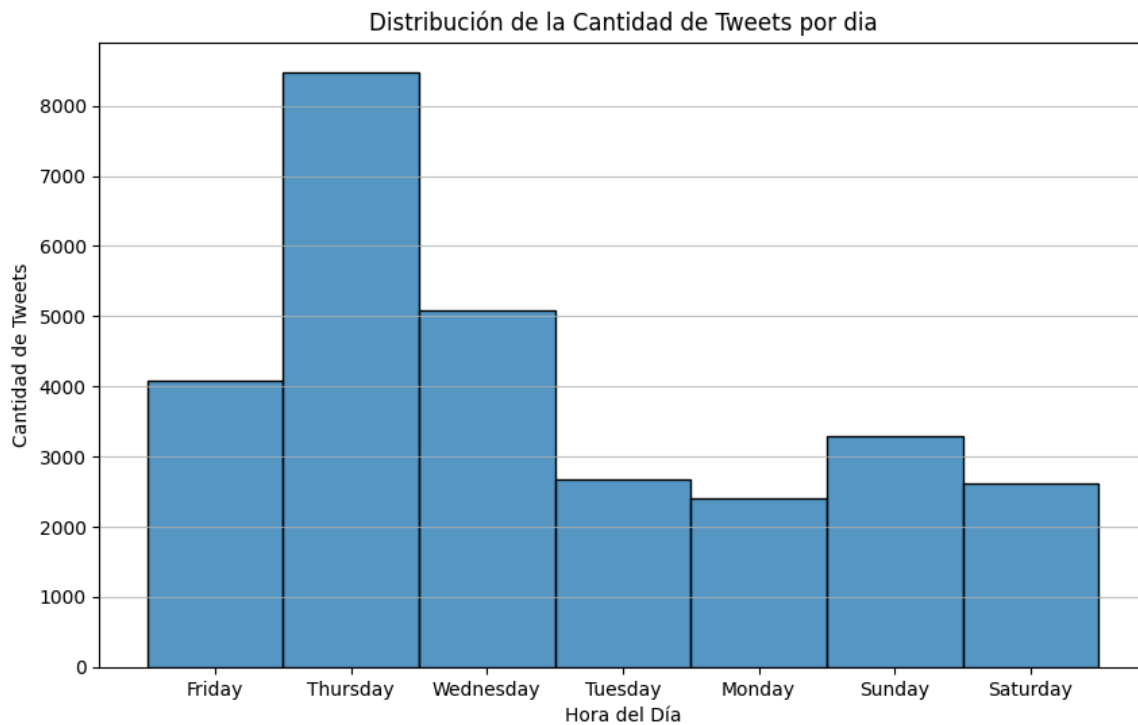
dtype: int64

Cantidad de tweets por hora:

	count
tweet_hour	
0	2229
1	1948
2	2050
3	1325
4	1194
5	1140
6	790
7	408
8	244
9	199
10	179
11	208
12	525
13	771
14	1185
15	1429
16	1481
17	1518
18	1614
19	1683
20	1487
21	1505
22	1660
23	1868

dtype: int64

```
# Histograma de la distribución de tweets por hora
plt.figure(figsize=(10, 6))
sns.histplot(tweets['tweet_day'], bins=24, kde=False) # Usamos 24 bins para cada hora
plt.title('Distribución de la Cantidad de Tweets por día')
plt.xlabel('Hora del Día')
plt.ylabel('Cantidad de Tweets')
plt.xticks(range(7)) # Asegurar que se muestren todas las horas si hay datos
plt.grid(axis='y', alpha=0.75)
plt.show()
```



```
login("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")
```

```
# =====
# LOGIN EN HUGGINGFACE
# =====
token = "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"
login(token)
os.environ["HF_TOKEN"] = token

# =====
# CARGAR CSV CORRECTAMENTE
# =====
df = pd.read_csv("usuarios_tw_09_11_2025.csv")

# La columna correcta es "text", no "tweet"
df = df.dropna(subset=["text"])

# Si hay textos vacíos, reemplazarlos
df['text'] = df['text'].fillna("").astype(str)

# =====
# CARGAR MODELO BETO CONTEXTUALIZADO
# =====
model_name = "piuba-bigdata/beto-contextualized-hate-speech"

tokenizer = AutoTokenizer.from_pretrained(model_name, token=token)
model = AutoModelForSequenceClassification.from_pretrained(model_name, token=token)
```

tokenizer_config.json: 100%

588/588 [00:00<00:00, 29.7kB/s]

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it

```
# =====
# FUNCIÓN DE CLASIFICACIÓN
# =====
def clasificar_contextualizado(text, context=""):
    entrada = f"{text} [SEP] {context}"

    inputs = tokenizer(
        entrada,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=256
    )

    with torch.no_grad():
        outputs = model(**inputs)

    logits = outputs.logits[0]
    preds = (logits > 0).cpu().numpy().tolist()

    labels = model.config.id2label
    return {labels[i]: preds[i] for i in range(len(preds))}

# =====
# APLICAR CLASIFICACIÓN A TODA LA COLUMNA "text"
# =====
df["predicciones"] = df["text"].apply(lambda t: clasificar_contextualizado(t))
```

```
# =====
# EXTRAER ETIQUETA BINARIA DE ODIO
# =====
def etiqueta_odio(dct):
    return 1 if any(dct.values()) else 0

df["odio_binario"] = df["predicciones"].apply(etiqueta_odio)

# =====
# MOSTRAR RESULTADO
# =====
```